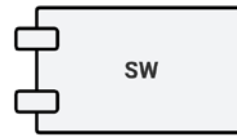


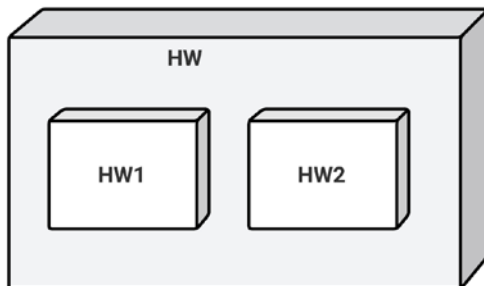
NODO

Recurso hardware.



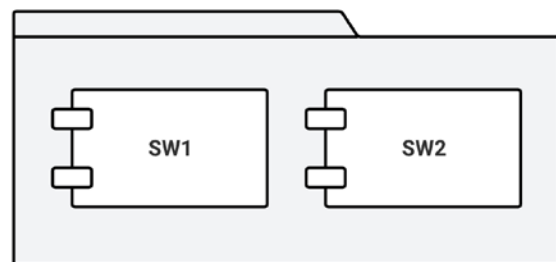
COMPONENTE

Recurso software.



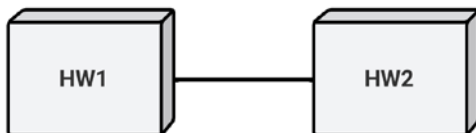
NODO COMPUESTO

Recursos hardware separados, pero incluidos dentro de un hardware más grande.



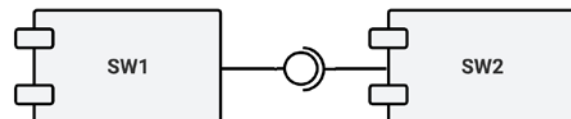
PAQUETE CON COMPONENTES

Conjunto de varios recursos software ubicados dentro de un mismo módulo.



CONEXIÓN ENTRE NODOS

Asociación mediante la cual los nodos transfieren información.



Dos componentes se comunican a través de una **interfaz**.

SW1 provee la **interfaz**, SW2 la usa.



BASE DE DATOS

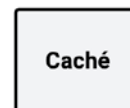
Asegura la persistencia de información.



LOAD BALANCER o BALANCEADOR DE CARGA

Recibe solicitudes y las distribuye entre los servidores, evitando cuellos de botella.

Provee escalabilidad y performance.



MEMORIA CACHE

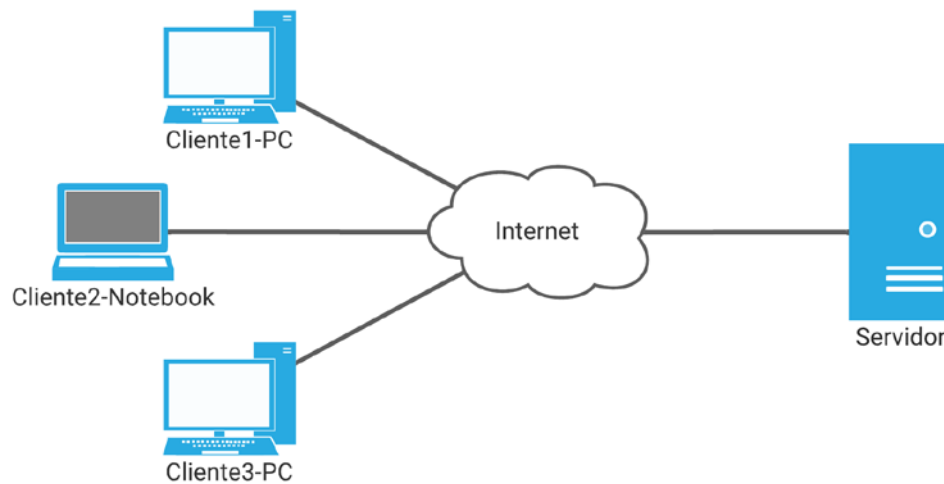
Memoria rápida que agiliza búsquedas y operaciones.

Provee performance.

ARQUITECTURA CLIENTE–SERVIDOR

- Un **servidor** provee servicios mediante una interfaz.
- Un **cliente** usa esos servicios como parte de su operación.
- Los clientes pueden establecer sesiones en el servidor, que mantendrá el estado de los clientes conectados.

Esquema:



Ventajas:

- ▲ Modificabilidad → los cambios de funcionalidades son centralizados.
- ▲ Testeabilidad.
- ▲ Mantenibilidad.

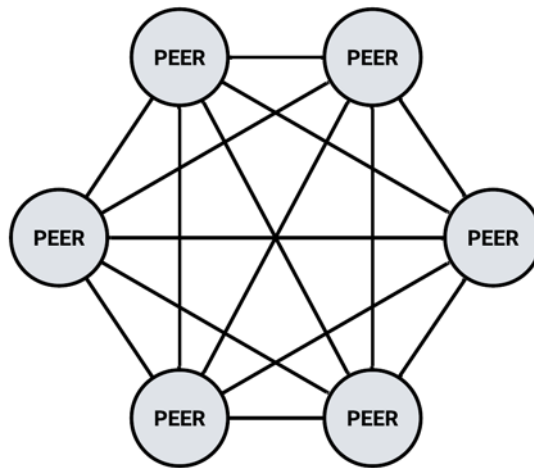
Desventajas:

- ▼ Performance → el servidor puede ser un cuello de botella.
- ▼ Disponibilidad → el servidor es un punto único de falla.

ARQUITECTURA PEER-TO-PEER (P2P)

- Los **nodos** actúan como clientes y servidores a la vez, simultáneamente.
- Cualquier nodo puede comunicarse con cualquier otro nodo, sin necesidad de utilizar un servidor central.
- Los nodos suelen localizarse entre sí mediante el intercambio automático de listas entre compañeros conocidos.
- El objetivo primordial es compartir recursos.

Esquema:



Ventajas:

▲ Escalabilidad → instanciando más nodos.

▲ Tolerancia a Fallos → si un nodo no funciona, la BD sigue trabajando con los nodos restantes.

Ventaja/Desventaja:

Disponibilidad → a mayor cantidad de nodos que comparte la misma información, mayor disponibilidad. Si no hay nodos que compartan la información, la disponibilidad será nula.

ARQUITECTURA POR CAPAS

- Los componentes de la arquitectura están organizados en capas, donde cada capa cumple un rol y tiene una responsabilidad específica.
- La comunicación entre las capas es mediante interfaces conocidas.

Ventajas:

- ▲ Cohesión → cada capa está abocada a resolver una sola cosa.
- ▲ Testeabilidad → es más fácil testear cada cosa por separado.
- ▲ Performance.
- ▲ Escalabilidad.

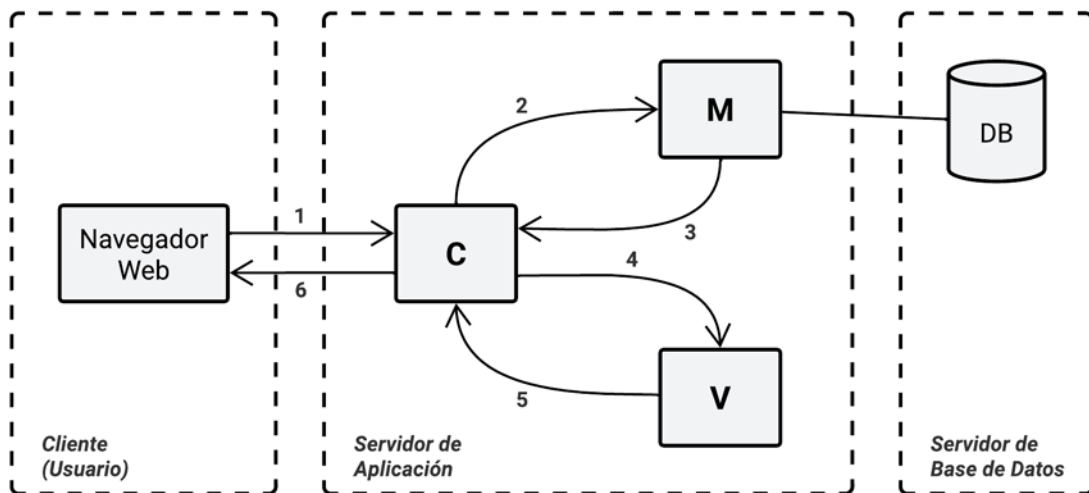
Desventaja:

- ▼ Acoplamiento → cada capa desconoce el funcionamiento de las otras.

PATRÓN MVC

- Separa al sistema en:
 - **Modelo** → encargado del almacenamiento de datos.
 - **Vista** → encargado de la presentación de los datos.
 - **Controlador** → encargado del procesamiento.
- La conexión entre el modelo y sus vistas es dinámica, produciéndose en *runtime*.
- Modo de Trabajo:
 1. El usuario envía una solicitud al controlador.
 2. El controlador le solicita los datos al modelo.
 3. El modelo le devuelve los datos.
 4. El controlador selecciona una vista.
 5. Se obtiene la vista.
 6. El controlador le devuelve al cliente la vista con los datos del modelo.

Esquema¹:



Ventajas:

- ▲ Cohesión → cada capa está abocada a resolver una sola cosa.
- ▲ Testeabilidad → las capas se pueden testear individualmente.
- ▲ Mantenibilidad → las capas se pueden modificar por separado sin problema.
- ▲ Performance.
- ▲ Equipo de Desarrollo.

Desventaja:

- ▼ Acoplamiento → cada capa desconoce el funcionamiento de las otras.

¹ En el esquema, la vista aparece generada en el servidor. Sin embargo, la vista también se puede resolver del lado del cliente.